

OBJECT ORIENTED SYSTEMS (OOS) — 2025

QUESTION 3 COMPLETE DETAILED SOLUTION

Introduction

Question 3 of the 2025 Object Oriented Systems examination belongs to CO3 and focuses on some of the most advanced and intellectually important areas of Java programming  .

This question mainly covers:

-  Java Reflection
-  Runtime Introspection
-  Dynamic Method Invocation
-  Generic Programming
-  Generic Methods
-  Type Parameterization
-  Reflection on String and StringBuffer

These concepts are extremely important because they form the backbone of enterprise frameworks, dependency injection systems, testing frameworks, IDE development, serialization systems, ORM frameworks, and runtime code analysis .

Technologies such as Spring Framework, Hibernate, JUnit, JavaBeans, and many modern frameworks internally use Reflection API, dynamic class loading, and generic programming extensively. Thus this question tests whether the student truly understands runtime behavior, metadata inspection, dynamic method invocation, and reusable type-safe programming .

QUESTION 3(a)

 **What special purpose is achieved by Java Reflection?**

 Answer

Java Reflection is a powerful feature that allows a Java program to inspect classes dynamically, analyze methods and fields, access constructors, invoke methods, and manipulate objects during runtime .

In simple words, Reflection enables a Java program to examine its own structure while executing. This capability is known as **runtime introspection**.

Why Reflection is Needed

Normally Java programs know class names, methods, and object structure during compilation. However, some applications need to load classes dynamically, invoke unknown methods, inspect metadata, and manipulate objects during runtime. Reflection solves this problem .

Special Purposes Achieved by Reflection

Reflection achieves several important purposes:

Purpose	Explanation
Runtime Introspection	Examine class structure dynamically
Dynamic Method Invocation	Invoke methods during runtime
Dynamic Object Creation	Create objects without explicit class references
Access Private Members	Access hidden fields/methods
Framework Development	Build flexible reusable frameworks

☀ Real-world Uses of Reflection

Reflection is heavily used in Spring Framework, Hibernate, JUnit, IDEs, Debuggers, Serialization systems, and Dependency Injection systems. For example, Spring automatically injects objects using Reflection.

💻 Reflection Example

```
class Student {
    void display() {
        System.out.println("Student Method");
    }
}

class Main {
    public static void main(String args[]) throws Exception {
        Class c = Class.forName("Student");
        Object obj = c.newInstance();

        java.lang.reflect.Method m = c.getMethod("display");
        m.invoke(obj);
    }
}
```

💻 Output

```
Student Method
```

☀ Deep Conceptual Analysis

The statement `Class.forName()` loads the class dynamically during runtime. Then `getMethod()` retrieves method metadata. Finally, `invoke()` executes the method dynamically. Thus method execution becomes runtime-controlled instead of compile-time controlled 🚀.

☀ Advantages & Disadvantages

- **Advantages:** Provides flexibility, runtime adaptability, dynamic loading, framework extensibility, and powerful debugging capabilities.
- **Disadvantages:** May reduce performance, break encapsulation, increase security risks, and complicate debugging. Thus excessive usage should be avoided.

Conclusion

Java Reflection provides runtime introspection and dynamic manipulation capabilities that allow programs to inspect and control classes, methods, and objects dynamically .

QUESTION 3(b)

? Write suitable code snippets to invoke the `charAt()`, `substring(int,int)`, and `indexOf()` methods of `String` class using reflection. Hence show the output with proper examples.

Answer

Using Java Reflection, methods of the `String` class can be dynamically located and invoked during runtime . Instead of directly calling methods normally, Reflection allows runtime method discovery, dynamic execution, and flexible method invocation.

Reflection Invocation Process

The process generally involves:

1. Load class metadata using `Class.forName()` or `.class`
2. Retrieve method using `getMethod()`
3. Invoke method using `invoke()`

Program for `charAt()`

```
import java.lang.reflect.*;

class Main {
    public static void main(String args[]) throws Exception {
        String s = "Java";
        Class c = String.class;

        Method m = c.getMethod("charAt", int.class);
        Object result = m.invoke(s, 2);

        System.out.println(result);
    }
}
```

Output

v

☀ **Explanation:** The method `charAt(2)` returns the character at index 2. Thus, `v` gets printed.

🖥 **Program for substring(int,int)**

```
import java.lang.reflect.*;

class Main {
    public static void main(String args[]) throws Exception {
        String s = "Programming";
        Class c = String.class;

        Method m = c.getMethod("substring", int.class, int.class);
        Object result = m.invoke(s, 3, 8);

        System.out.println(result);
    }
}
```

🖥 **Output**

gramm

☀ **Explanation:** The substring begins from index 3 and continues till index 7. Thus output becomes `gramm`.

🖥 **Program for indexOf()**

```
import java.lang.reflect.*;

class Main {
    public static void main(String args[]) throws Exception {
        String s = "Reflection";
        Class c = String.class;

        Method m = c.getMethod("indexOf", String.class);
        Object result = m.invoke(s, "lec");

        System.out.println(result);
    }
}
```

🖥 **Output**

3

🌟 Deep Conceptual Analysis

Reflection dynamically retrieves method metadata, parameter types, return type, and invocation logic. The JVM internally resolves methods using runtime class metadata, method tables, and reflection APIs 🚀.

🎯 Conclusion

Reflection enables dynamic invocation of String methods during runtime and provides flexible runtime method execution capabilities 🚀.

📄 QUESTION 3(c)

❓ **Define a generic class having 3 overloaded versions of a generic method. One of them should be static. Now show with a suitable program how to invoke those methods from main().**

✅ Answer

Generic Programming allows classes and methods to operate on different datatypes without rewriting logic repeatedly 🚀. Generics improve code reusability, type safety, and flexibility.

🌟 Why Generics are Needed

Without generics, separate methods are required for `int`, `float`, `String`, `character`, objects, etc. Generics eliminate this duplication.

💻 Generic Class with Overloaded Generic Methods

```
class Demo<T> {
    T data;

    Demo(T data) {
        this.data = data;
    }

    // Generic Method 1
    <T> void show(T value) {
        System.out.println("Generic Method 1: " + value);
    }

    // Generic Method 2
    <T,U> void show(T x, U y) {
        System.out.println("Generic Method 2: " + x + " " + y);
    }

    // Static Generic Method
    static <T> void display(T value) {
```

```
        System.out.println("Static Generic Method: " + value);
    }
}

class Main {
    public static void main(String args[]) {
        Demo<Integer> d = new Demo<>(10);

        d.show(100);
        d.show("Java", 50);

        Demo.display(3.14);
    }
}
```

Output

```
Generic Method 1: 100
Generic Method 2: Java 50
Static Generic Method: 3.14
```

Deep Conceptual Analysis

The generic methods dynamically adapt to the datatype provided during invocation. Thus the same method handles integers, strings, doubles, and mixed datatypes. This reduces redundancy significantly 🚀.

Why Static Generic Methods are Allowed

Static methods belong to the class level. Thus they define independent generic parameters explicitly (e.g., `static <T> void display(T value)`), completely independent of the class's type parameters.

Conclusion

Generic methods provide reusable type-safe programming and significantly improve software flexibility and maintainability 🚀.

QUESTION 3(d)

? Define a generic class with a member method `highest_occurrence()` that takes arrays `A` of different datatypes (`int`, `float`, `char`) as input and finds the highest occurring element.

Answer (Generic `highest_occurrence()` Program)

This question combines generic programming, arrays, frequency counting, and reusable method design 🚀. The goal is to accept arrays of different datatypes, count occurrences, and determine the highest occurring element.

Generic Program

```
class Frequency<T> {
    void highest_occurrence(T arr[]) {
        int maxCount = 0;
        T maxElement = null;

        for(int i = 0; i < arr.length; i++) {
            int count = 1;
            for(int j = i + 1; j < arr.length; j++) {
                if(arr[i].equals(arr[j])) {
                    count++;
                }
            }

            if(count > maxCount) {
                maxCount = count;
                maxElement = arr[i];
            }
        }

        System.out.println("Highest occurring element: " + maxElement);
    }
}

class Main {
    public static void main(String args[]) {
        // Integer Array
        Frequency<Integer> f1 = new Frequency<>();
        Integer a[] = {1, 2, 1, 3, 4, 3, 2, 1};
        f1.highest_occurrence(a);

        // Character Array
        Frequency<Character> f2 = new Frequency<>();
        Character c[] = {'a', 'b', 'a', 'c', 'a'};
        f2.highest_occurrence(c);
    }
}
```

Output

```
Highest occurring element: 1
Highest occurring element: a
```

Deep Conceptual Analysis

The generic type `<T>` allows the same logic to work for `Integer`, `Character`, `Float`, and other object datatypes. Thus code reuse improves significantly 🚀.

Conclusion

Generics enable reusable datatype-independent programming and simplify frequency analysis across multiple datatypes 🚀.

📄 QUESTION 3(d) — OR PART

? **Invoke any 4 methods of `StringBuffer` class (which are not available for `String` class) using reflection. Hence show the output of each method with proper example.**

✅ Answer & Introduction

The `StringBuffer` class in Java is a mutable sequence of characters 🚀. Unlike the `String` class, whose objects are immutable, `StringBuffer` objects can be modified dynamically without creating new objects repeatedly. This makes `StringBuffer` extremely useful in text editors, dynamic string manipulation, compiler design, logging systems, and high-performance applications 🌐.

The question specifically asks to invoke methods of `StringBuffer` using **Reflection API**. Reflection allows a Java program to inspect classes dynamically, retrieve methods, and invoke them during runtime. This demonstrates runtime introspection, dynamic method invocation, and advanced JVM capabilities 🗨️.

🌟 Methods Chosen

The following four methods of `StringBuffer` will be invoked using reflection. These methods are special because they modify the same object dynamically and are not directly available in immutable `String` objects 🚀.

Method	Purpose
<code>append()</code>	Add text at end
<code>reverse()</code>	Reverse characters
<code>delete()</code>	Remove characters
<code>insert()</code>	Insert text at position

📄 1 Invoking `append()` Using Reflection

🌟 **Purpose:** The method `append()` adds new characters at the end of the existing buffer. Unlike `String` concatenation, no new object is created; the existing object gets modified. This improves performance significantly ⚡.

💻 Java Program

```
import java.lang.reflect.*;

class Main {
    public static void main(String args[]) throws Exception {
        StringBuffer sb = new StringBuffer("Java");
        Class c = StringBuffer.class;
```

```
Method m = c.getMethod("append", String.class);
m.invoke(sb, " Programming");

System.out.println(sb);
}
}
```

Output

```
Java Programming
```

2 Invoking reverse() Using Reflection

🔦 **Purpose:** The method `reverse()` reverses the character sequence inside the same buffer object 🚀.

Java Program

```
import java.lang.reflect.*;

class Main {
    public static void main(String args[]) throws Exception {
        StringBuffer sb = new StringBuffer("Object");
        Class c = StringBuffer.class;

        Method m = c.getMethod("reverse");
        m.invoke(sb);

        System.out.println(sb);
    }
}
```

Output

```
tcejbO
```

3 Invoking delete() Using Reflection

🔦 **Purpose:** The method `delete()` removes characters between specified indexes.

Java Program

```
import java.lang.reflect.*;
```

```
class Main {
    public static void main(String args[]) throws Exception {
        StringBuffer sb = new StringBuffer("Programming");
        Class c = StringBuffer.class;

        Method m = c.getMethod("delete", int.class, int.class);
        m.invoke(sb, 3, 6);

        System.out.println(sb);
    }
}
```

Output

Promming

(Characters from index 3 to 5 (*gra*) get deleted).



Invoking insert() Using Reflection

🌟 **Purpose:** The method `insert()` adds characters at a specific position inside the buffer.



Java Program

```
import java.lang.reflect.*;

class Main {
    public static void main(String args[]) throws Exception {
        StringBuffer sb = new StringBuffer("Jav");
        Class c = StringBuffer.class;

        Method m = c.getMethod("insert", int.class, String.class);
        m.invoke(sb, 3, "a");

        System.out.println(sb);
    }
}
```

Output

Java

🌟 Why Reflection is Important Here

Normally methods are invoked directly (e.g., `sb.append("Text")`). However, Reflection enables runtime method discovery, dynamic invocation, flexible framework behavior, and metadata-driven execution 🚀. This capability is heavily used in Spring Framework, Hibernate, JUnit, IDEs, and serialization libraries 🧠.

☀️ Reflection Invocation Steps

Every reflection-based invocation follows:

1. `Class.forName()` / `.class` : Load metadata
2. `getMethod()` : Retrieve method
3. `invoke()` : Execute dynamically

☀️ Difference Between String and StringBuffer

Feature	String	StringBuffer
Mutability	Immutable	Mutable
Performance	Slower for repeated changes	Faster
Thread Safety	No	Yes
Memory Efficiency	Lower	Higher

☀️ JVM Internal Working

Internally Reflection uses runtime metadata tables, method descriptors, dynamic dispatch systems, and class loaders. The JVM dynamically resolves parameter types, return types, and execution targets during runtime 🚀.